

HireGenie AI

Autonomous Recruitment & Talent Intelligence Platform (TalentOS)

Complete Code Architecture & Project Overview
Generated: August 20, 2026 at 01:06 PM
GitHub: github.com/HelloRohit26/HireGenie-AI-Recruitment-Platform

Dimension	Detail
Platform Type	Enterprise-grade Autonomous Multi-Agent Recruitment OS
Backend	FastAPI + Python 3.11 + SQLAlchemy + Celery + Redis
Frontend	React 18 + TypeScript + Vite + Tailwind CSS + Framer Motion
AI Engine	Google Gemini AI + Canonical Skill Normalizer (400+ skills)
Database	SQLite (Dev) / PostgreSQL 16 (Production)
Voice Interview	WebRTC Speech-to-Speech + Twilio Telephonic Dial-out
Trust & Safety	RBAC, Audit Middleware, Bias/Fairness (4/5ths rule), Explainable AI
Task Queue	Celery with Redis broker, idempotent retry, exponential backoff
API Endpoints	17 RESTful route controllers under /api/v1/
Frontend Pages	21 full portal screens (Recruiter + Candidate dual portals)
Test Suite	20 end-to-end verification scripts

Table of Contents

- 1. Executive Overview & Platform Vision
- 2. System Architecture Diagram
- 3. Complete Directory Structure
- 4. Backend Architecture Deep Dive
 - 4.1 FastAPI Application Entrypoint (main.py)
 - 4.2 API Layer & Route Controllers (17 Endpoints)
 - 4.3 Database Models & Relational Schema (14 Tables)
 - 4.4 Service Layer & AI Pipelines (15 Services)
 - 4.5 Durable Celery Worker Task Queue
 - 4.6 Core Infrastructure & Security
 - 4.7 Pydantic Schemas & Validation
- 5. Frontend Architecture Deep Dive
 - 5.1 Application Router & Auth Guards (App.tsx)
 - 5.2 Page Components (21 Screens)
 - 5.3 Reusable Component Library (33 Components)
 - 5.4 API Service Layer (8 Typed Clients)
 - 5.5 Design System & Motion Engine
- 6. End-to-End Autonomous Hiring Pipeline
- 7. Trust & Safety Layer
- 8. Deployment & DevOps
- 9. Test Suite & Quality Assurance
- 10. Environment Variables Reference

1. Executive Overview & Platform Vision

HireGenie AI is a production-grade autonomous recruitment platform that eliminates manual recruiter intervention across the entire hiring lifecycle. It orchestrates a multi-agent AI pipeline spanning JD Intelligence, resume parsing with canonical skill normalization (400+ technology synonyms), multi-criteria candidate screening and dynamic job-scoped ranking, real-time WebRTC and Twilio telephonic voice-to-voice AI interviews, deterministic scorecard evaluation, and automated offer letter dispatch.

Key Differentiators:

- â■¢ **Zero-Intervention Pipeline:** From job posting to offer letter, no manual recruiter step is required.
- â■¢ **Dual-Portal Ecosystem:** TalentOS Recruiter Command Center + Candidate Discovery & Interview Portal.
- â■¢ **Glassmorphism UI Engine:** Cyber-Obsidian dark mode (#03040a) and Warm Off-White light mode with Framer Motion spring physics.
- â■¢ **Trust & Safety Layer:** Disparate Impact Ratio bias detection (4/5ths rule), Explainable AI with human-in-the-loop override, and immutable audit trail.
- â■¢ **Durable Job Queue:** Celery workers with idempotent retry and exponential backoff for screening, evaluation, and email delivery.

2. System Architecture Diagram

The platform follows a clean three-tier architecture with clear separation of concerns:

Tier	Technology	Responsibility
Presentation	React 18 + TypeScript + Vite	21-screen dual-portal SPA with glassmorphism design, Framer Motion, WebRTC audio
API Gateway	FastAPI + Pydantic v2	17 RESTful route controllers, RBAC middleware, CORS, audit logging
Business Logic	15 service modules + Gemini AI	AI screening pipeline, evaluation engine, conversational interview, communication agent
Data Layer	SQLAlchemy ORM + 14 tables	Relational models with cascading relationships, JSON fields for AI payloads
Task Queue	Celery + Redis	Durable async workers for screening, evaluation, email with retry policies
Vector Store	ChromaDB	Resume embedding storage for semantic skill matching

End-to-End Data Flow:

Recruiter Posts Job (JD Intelligence) --> Candidate Applies (Resume Upload) --> AI Resume Parser (Gemini + Canonical Skill Normalizer) --> Multi-Criteria Screening Pipeline (Score + Rank) --> Automated Shortlist Invitation (Email Agent) --> Real-Time Voice Interview (WebRTC/Twilio) --> Deterministic Evaluation Scorecard --> Recruiter Dossier & Human-in-the-Loop Override --> Autonomous Offer Letter Generation

3. Complete Directory Structure

```
HireGenie-AI-Recruitment-Platform/
+-- README.md                # Full documentation & quickstart
+-- design.md                # TalentOS UI design system spec
+-- MASTER_FRONTEND_PROMPT.md # UI screen workflow specifications
+-- docker-compose.production.example.yml # Production Docker orchestration
+-- .gitignore               # Git exclusion rules
+-- docs/                    # 46 system audit & batch reports
+-- stitch_hiregenie_ai_talentos/ # Original UI mockup inventory (37 screens)
|
+-- backend/
|   +-- Dockerfile           # Production container build
|   +-- docker-compose.yml   # Local dev compose (PG + Redis + Celery)
|   +-- requirements.txt     # Python dependencies
|   +-- alembic/             # Database migration configs
|   +-- tests/               # 20 E2E verification scripts
|   +-- app/
|       +-- main.py          # FastAPI entrypoint & startup
|       +-- api/v1/
|           | +-- api.py     # Central router registry
|           | +-- endpoints/ # 17 RESTful route controllers
|       +-- core/            # Security, RBAC, config, Gemini client
|       +-- db/              # Database session & base models
|       +-- models/          # 7 SQLAlchemy model files (14 tables)
|       +-- schemas/         # 11 Pydantic validation schemas
|       +-- services/        # 15 business logic & AI services
|       +-- workers/         # Celery app, tasks & dispatcher
|       +-- tools/           # Voice tools (Sarvam AI TTS)
|       +-- integrations/    # External integration base
|
+-- frontend/
+-- index.html               # SPA entry HTML
+-- package.json             # Node dependencies
+-- vite.config.ts           # Vite build config
+-- tsconfig.json            # TypeScript compiler settings
+-- tests/                   # Frontend test scripts
+-- src/
+--   App.tsx                # Main router & auth guards
+--   index.css              # Design tokens & animations
+--   main.tsx               # React DOM entry
+--   pages/                 # 21 full portal page components
+--   components/            # 10 component directories (33 components)
+--   services/              # 8 typed API client services
+--   context/               # Theme & auth context providers
+--   types/                 # TypeScript interfaces
+--   hooks/                 # Custom React hooks
+--   utils/                 # Utility functions
+--   data/                  # Mock data for dev
```

4. Backend Architecture Deep Dive

4.1 FastAPI Application Entrypoint

The `main.py` (237 lines) initializes the FastAPI application with: CORS middleware for frontend origins, audit logging middleware on every request, startup auto-migration of 40+ column additions across jobs/users/applications tables, seed data creation (Admin + Recruiter + Sample Job with screening questions), and mounting of the central API router at `/api/v1`.

4.2 API Layer & Route Controllers (17 Endpoints)

Route	Module	Responsibility	LOC
/api/v1/auth	auth.py	JWT login, registration, role-based tokens, password hashing	3,568
/api/v1/jobs	jobs.py	Job CRUD, 5-step wizard, JD Intelligence, search & filter	14,737
/api/v1/candidate	candidate.py	Application submission, resume upload, status tracking, apply flow	29,451
/api/v1/recruiter	recruiter.py	Mass screening trigger, candidate dossier, shortlist management	12,179
/api/v1/interview	interview.py	Session creation, invitation flow, interview lifecycle	17,452
/api/v1/interview (WS)	voice_ws.py	WebSocket voice stream, Sarvam AI TTS, real-time transcription	17,060
/api/v1/hiring	hiring.py	Final hiring decisions, offer generation, acceptance/decline	20,369
/api/v1/explainability	explainability.py	AI decision explanations, recruiter override with audit trail	5,499
/api/v1/fairness	fairness.py	Disparate impact ratio, demographic parity, bias flags	2,420
/api/v1/jd	jd_intelligence.py	AI-powered skill extraction, JD quality scoring	2,619
/api/v1/scheduling	scheduling.py	Interview slot management, calendar coordination	3,103
/api/v1/communication	communication.py	Email templates, multi-channel dispatch, delivery tracking	3,690
/api/v1/analytics	analytics.py	Pipeline metrics, funnel analytics, source performance	2,292
/api/v1/audit	audit.py	Immutable audit log query, export, filtering	3,185
/api/v1/failures	failures.py	Failed task queue inspection and manual retry triggers	1,818
/api/v1/integrations	integrations.py	External service connection management	1,576
/api/v1/admin	admin.py	System admin operations, data cleaning, health checks	2,371

4.3 Database Models & Relational Schema

All database models derive from SQLAlchemy **Base** (in app/db/base.py). The primary models are defined in **models.py** (430 lines), with feature-specific models in dedicated files. The schema contains 14 relational tables with cascading foreign key relationships:

Table	Model File	Key Fields & Purpose
users	models.py	id, full_name, email, hashed_password, role (ADMIN/RECRUITER/CANDIDATE), phone, last_login
jobs	models.py	35+ columns: title, company, salary fields, JD intelligence (extracted_skills, must_have, weights, jd_quality_score), interview config, screening config
screening_questions	models.py	job_id FK, question_text, category, weight, is_required
resumes	models.py	candidate_id FK, file_path, raw_text, parsed_skills (JSON), experience_years, vector_id
candidate_applications	models.py	candidate_id, job_id, resume_id FKs, status (18 states), overall_match_score, rank, magic_token
agent_telemetry	models.py	application_id FK, agent_name, status, duration_ms, details (JSON)
screening_answers	models.py	application_id, question_id FKs, answer_text, ai_score, ai_feedback
interview_invitations	models.py	invitation_token, status (8 states), interview_mode, expires_at, viewed_at
interview_sessions	models.py	session_token, status (8 states), transcript (JSON), elapsed_seconds
interview_evaluations	models.py	technical/problem_solving/communication/role_fit/overall scores, recommendation, strengths, gaps
hiring_decisions	models.py	decision (HIRED/REJECTED), decided_by FK, reason
job_offers	models.py	offer_token, status (OFFERED/ACCEPTED/DECLINED), compensation, expires_at
audit_logs	audit.py	actor_type, actor_id, action, resource, ip_address, details (JSON)
ai_explanations	explainability.py	explanation_type, confidence, reasoning, feature_contributions (JSON)
recruiter_overrides	explainability.py	original_decision, override_decision, justification
fairness_reports	fairness.py	demographic_group, metric_name, metric_value, is_compliant
bias_flags	fairness.py	bias_type, severity, description
communication_logs	communication.py	stage, channel, recipient_email, subject, body, delivery_status, retry_count
interview_schedules	scheduling.py	scheduled_start, scheduled_end, status, calendar_event_id
failed_tasks	failure_queue.py	task_type, payload (JSON), error_message, retry_count, status

4.4 Service Layer & AI Pipelines (15 Services)

Service	Bytes	Purpose & AI Capabilities
screening_pipeline.py	19,193	Core autonomous screening engine: canonical skill normalization (400+ synonyms), multi-criteria scoring (skills, experience, education, projects), Gemini AI reasoning, dynamic job-scoped ranking (#1, #2, #3...), auto-shortlist with email dispatch
evaluation_service.py	13,000	Post-interview deterministic scorecard: technical, problem-solving, communication, role-fit scores. Gemini AI generates evidence-backed recommendation (STRONG_HIRE/HIRE/CONSIDER/NO_HIRE)
interview_conversational_engine.py	14,543	Real-time voice interview AI: dynamic question generation, conversational follow-ups, behavioral probes, technical deep-dives adapted to candidate responses
analytics_service.py	16,416	Comprehensive recruitment analytics: pipeline funnel metrics, time-to-hire, source performance, skill demand analysis, hiring velocity trends
communication_agent.py	10,811	Multi-stage email orchestration: shortlist notifications, interview invitations with magic links, offer letters. Template engine with personalized variables
fairness_service.py	10,741	Disparate Impact Ratio (4/5ths rule) analysis, demographic parity scoring, bias flag detection, compliance reporting
resume_parser.py	10,466	PDF/DOCX/TXT extraction via PyMuPDF, Gemini AI structured parsing for skills, experience, education, projects with canonical normalization
scheduling_service.py	10,045	Interview slot management, availability coordination, calendar integration, automatic scheduling
email_provider.py	8,180	Multi-provider email delivery: SMTP, SendGrid, Resend. Auto-detection of configured provider with fallback chain
jd_intelligence_service.py	5,288	AI-powered job description analysis: skill extraction, must-have vs nice-to-have classification, quality scoring
retry_service.py	6,746	Exponential backoff retry logic, dead-letter queue management, failure categorization
audit_service.py	3,512	Immutable audit trail recording with actor identification, IP tracking, and resource versioning
vector_service.py	2,896	ChromaDB vector store for resume embeddings, semantic skill similarity search
voice_service.py	1,448	Voice session management, audio recording coordination
communication_service.py	1,533	Helper layer bridging screening pipeline to communication agent for shortlist emails

4.5 Durable Celery Worker Task Queue

The task queue ensures reliability via idempotent workers with exponential backoff retries:

File	Purpose
workers/celery_app.py	Celery application factory with Redis broker, eager mode fallback for dev, configurable durable queue toggle
workers/tasks.py	3 durable tasks: screen_application_task (AI screening), evaluate_interview_task (post-interview evaluation), send_email_task (email delivery). All idempotent with state machine transitions
workers/dispatcher.py	Task dispatch interface with Redis availability checks, durable queue enforcement, explicit failure reporting

4.6 Core Infrastructure & Security

Module	Purpose
core/config.py	Pydantic BaseSettings with 35+ environment variables: DB, JWT, Gemini, Sarvam AI, SMTP, SendGrid, Resend, LiveKit
core/rbac.py	Role-Based Access Control dependency injection: JWT extraction, user validation, role enforcement (ADMIN/RECRUITER/CANDIDATE)
core/auth.py	JWT token creation and decoding with HS256 algorithm and configurable expiry
core/security.py	Password hashing with bcrypt and verification utilities
core/audit_middleware.py	FastAPI middleware logging every HTTP request: method, path, status, duration, IP, user
core/skill_normalizer.py	400+ technology synonym dictionary mapping (e.g., ReactJS/React.js/react --> React, Postgres/PostgreSQL --> PostgreSQL)
core/gemini.py	Google Gemini AI client initialization with API key
core/llm.py	LLM abstraction layer for model selection
core/logger.py	Centralized logging configuration
db/session.py	SQLAlchemy engine factory with SQLite/PostgreSQL auto-detection, production safety guards
db/base.py	SQLAlchemy declarative base for all ORM models

5. Frontend Architecture Deep Dive

5.1 Application Router & Auth Guards (App.tsx)

The **App.tsx** (222 lines) implements a custom client-side router with query-parameter-based navigation, localStorage token persistence, role-based route guards (recruiter routes require JWT + RECRUITER/ADMIN role), browser history API integration, and regex-based dynamic route matching for interview tokens and job IDs.

5.2 Page Components (21 Screens)

Page	Route	Portal	Bytes
EntryLandingPage	/	Public	9,219
RecruiterCommandCenter	/recruiter	Recruiter	10,839
RecruiterJobsPage	/recruiter/jobs	Recruiter	9,174
JobWorkspacePage	/recruiter/jobs/:id	Recruiter	10,836
CandidatesPage	/recruiter/candidates	Recruiter	12,627
AI ScreeningPage	/recruiter/screening	Recruiter	29,534
InterviewsPage	/recruiter/interviews	Recruiter	12,730
InsightsPage	/recruiter/insights	Recruiter	4,443
SettingsPage	/recruiter/settings	Recruiter	23,414
TrustSafetyPage	/recruiter/trust-safety	Recruiter	597
RecruiterOnboardingPage	/recruiter/onboarding	Recruiter	12,609
CandidateHomePage	/candidate	Candidate	9,464
CandidateJobsPage	/candidate/jobs	Candidate	7,322
CandidateJobDetailPage	/candidate/jobs/:id	Candidate	10,788
MyApplicationsPage	/candidate/applications	Candidate	7,483
CandidateOnboardingPage	/candidate/onboarding	Candidate	10,564
InterviewEntryPage	/interview/:token	Candidate	6,392
InterviewPrepPage	/interview/:token/prep	Candidate	34,922
VoiceInterviewRoomPage	/interview/:token/room	Candidate	19,937
OfferPortalPage	/offer/:token	Candidate	12,422
HelpPage	/help	Public	6,539

5.3 Reusable Component Library (33 Components)

Directory	Components	Purpose
components/auth/	SignInModal (23,764 bytes)	Dual-mode auth modal with recruiter/candidate role selection, JWT login, demo login
components/landing/	SharpNavyRainCanvas, SplineHeroBackground, InteractiveKnowledgeGraph	High-DPI canvas rain animation, Spline 3D hero, interactive knowledge graph visualization
components/layout/	RecruiterShell, RecruiterSidebar, RecruiterHeader, CandidateShell	Portal shells with responsive sidebars, navigation, theme toggle, user profile widgets
components/recruiter/	14 components: ActiveAgentOperations, RecruitmentPipeline, ExplainabilityPanel, FairnessReportPanel, AuditLogViewer, etc.	TalentOS dashboard widgets: agent ops, pipeline funnel, AI explainability, bias reports, failure queue
components/candidate/	ApplicationJourneyCard, ApplicationTimeline, ApplyModal, InterviewConsentModal, JobDiscoveryCard	Candidate journey visualization, application flow modals, job discovery cards
components/ui/	CommandPalette, InteractiveTiltCard, MetricCard, MobileDrawer, NotificationPanel, PageTransition	Reusable UI primitives: command palette (Cmd+K), 3D tilt cards, animated metrics, transitions

5.4 API Service Layer (8 Typed Clients)

Service	Bytes	Endpoints Consumed
apiClient.ts	3,475	Central Axios client with JWT Bearer token injection, base URL config, error interceptors
authService.ts	4,265	Login, register, token refresh, role detection, localStorage persistence
jobService.ts	9,514	Job CRUD, wizard creation, search/filter, status management, JD intelligence triggers
candidateService.ts	12,453	Application submission, resume upload, status tracking, interview invitation, dossier fetch
webrtcService.ts	22,083	WebRTC audio session management, peer connection, media stream handling, Sarvam AI TTS integration
screeningService.ts	2,422	Mass screening triggers, ranking queries, screening status polling
aiAgentService.ts	5,220	AI agent status monitoring, pipeline throughput metrics, telemetry queries
analyticsService.ts	1,384	Dashboard analytics, funnel metrics, time-series data

6. End-to-End Autonomous Hiring Pipeline

- â■¶ 1. **JD Intelligence:** Recruiter creates job via 5-step wizard. Gemini AI extracts skills, classifies must-have vs nice-to-have, assigns skill weights, and generates JD quality score.
- â■¶ 2. **Candidate Application:** Candidate discovers jobs, uploads resume (PDF/DOCX/TXT), optionally answers screening questions, and submits application.
- â■¶ 3. **Resume Parsing:** ResumeParser extracts text via PyMuPDF/python-docx, sends to Gemini AI for structured parsing (skills, experience, education, projects). SkillNormalizer maps 400+ synonyms to canonical forms.
- â■¶ 4. **AI Screening:** ScreeningPipeline computes multi-criteria match score: skill overlap (must-have/nice-to-have with weights), experience years, education level, project relevance. Gemini AI provides reasoning narrative.
- â■¶ 5. **Dynamic Ranking:** Job-scoped ranking engine recalculates positions (#1, #2, #3...) across ALL candidates for that job by descending match score. Cross-job isolation guaranteed.
- â■¶ 6. **Auto-Shortlist:** Candidates exceeding shortlist_threshold are auto-promoted to SHORTLISTED status. CommunicationAgent dispatches personalized email with magic interview link.
- â■¶ 7. **Voice Interview:** Candidate clicks magic link, completes interview prep page, enters real-time WebRTC voice room. AI conversational engine generates dynamic questions adapted to responses.
- â■¶ 8. **Evaluation:** EvaluationService generates deterministic scorecard: technical (0-100), problem-solving, communication, role-fit scores. Gemini AI produces evidence-backed recommendation.
- â■¶ 9. **Human Review:** Recruiter reviews candidate dossier with full transcript, audio scrubber, scorecard, and AI explanation. Optional human-in-the-loop override with mandatory justification.
- â■¶ 10. **Offer Dispatch:** Autonomous offer letter generation with compensation details, unique offer_token for acceptance portal. Candidate can accept or decline with reason.

7. Trust & Safety Layer

RBAC (Role-Based Access Control): Every endpoint is protected via JWT dependency injection. Three roles (ADMIN, RECRUITER, CANDIDATE) with granular permission checks. Recruiter routes blocked from candidate tokens.

Audit Middleware: AuditMiddleware logs every HTTP request with method, path, status code, response time (ms), client IP, and authenticated user. Immutable AuditLog table with export capabilities.

Explainable AI: Every AI screening decision includes feature_contributions JSON showing exactly how skills, experience, and education impacted the score. RecruiterOverride table tracks all human corrections.

Bias & Fairness Monitoring: FairnessService computes Disparate Impact Ratio (4/5ths rule) across demographic groups. BiasFlags raised when metrics fall below compliance thresholds. FairnessReports stored per analysis run.

Failure Queue & Retry: FailedTask table captures all worker errors with full payload and stack trace. RetryService implements exponential backoff ($2^n * 5s$) with configurable max_retries. Dead-letter queue for exhausted tasks.

8. Deployment & DevOps

Docker Compose (Local Development): PostgreSQL 16 + Redis 7 + FastAPI Backend + Celery Worker with health checks and shared volumes.

Docker Compose (Production): Redis + Backend (uvicorn) + Celery Worker with durable queue enabled, SQLite or PostgreSQL configurable via DATABASE_URL.

Dockerfile: Python 3.11-slim base, pip install from requirements.txt, uvicorn entrypoint on port 8000.

9. Test Suite & Quality Assurance

Test File	Coverage Area
test_real_final_hiring.py (22K)	Complete hiring lifecycle: job creation, application, screening, interview, evaluation, offer, acceptance
test_complete_journey_verification.py	End-to-end candidate journey from registration to hired status
test_real_voice_interview.py	WebRTC voice session creation, WebSocket streaming, transcription, evaluation
test_real_interview_evaluation.py	Post-interview scorecard generation, recommendation accuracy
test_candidate_journey_flow.py	Candidate portal flow: discover jobs, apply, track status, receive invitation
test_real_ai_screening.py	AI screening pipeline: skill matching, score computation, ranking verification
test_shortlist_email_invitation.py	Email dispatch, magic link generation, invitation status tracking
test_real_interview_invitation.py	Invitation lifecycle: creation, token validation, acceptance, expiry
test_durable_job_queue.py	Celery task dispatch, Redis broker, retry policies, eager mode fallback
test_create_job_wizard.py	5-step job creation wizard with JD intelligence and screening question configuration
test_real_email_delivery.py	Multi-provider email delivery: SMTP, SendGrid, Resend with failure handling
test_real_candidate_auth.py	JWT authentication: registration, login, token validation, role enforcement
test_real_agent_pipeline.py	Multi-agent pipeline telemetry: agent status transitions, duration tracking
run_full_api_audit.py (49K)	Comprehensive automated audit of all 17 API endpoint groups with request/response validation

10. Environment Variables Reference

Variable	Default	Purpose
DATABASE_URL	sqlite:///./hiregenie.db	SQLAlchemy connection string (PostgreSQL for production)
SECRET_KEY	(change)	JWT signing secret key
GEMINI_API_KEY	(required)	Google Gemini AI API key for all AI reasoning
SARVAM_API_KEY		Sarvam AI TTS API key for Hindi/English voice synthesis
LIVEKIT_URL		LiveKit server URL for WebRTC room management
LIVEKIT_API_KEY/SECRET		LiveKit authentication credentials
REDIS_URL	redis://localhost:6379/0	Redis broker URL for Celery task queue
DURABLE_QUEUE_ENABLED	false	Enable durable Celery workers (requires Redis)
EMAIL_PROVIDER	auto	Email provider: auto, smtp, sendgrid, or resend
SMTP_HOST/PORT/USER/PASSWORD		SMTP server configuration for email delivery
SENDGRID_API_KEY		SendGrid API key for transactional email
RESEND_API_KEY		Resend API key for developer email service
CORS_ORIGINS	localhost:5173,3000	Comma-separated allowed frontend origins